

TRƯỜNG / KHOA / BỘ MÔN
BÀI TẬP LỚN MÔN KIẾN TRÚC VÀ THIẾT KẾ PHẦN MỀM
Đề tài: Phân tích, mô hình hóa và đánh giá hệ thống Base Backend API
Phiên bản báo cáo chuyên sâu 30-40 trang

Giảng viên hướng dẫn	
Sinh viên thực hiện	
Mã sinh viên	
Lớp	
Nhóm	
Repository khảo sát	TCNS-DCD-BACK
Sản phẩm khảo sát	Base Backend API - hệ thống quản lý nội bộ TCNS
Ngày lập báo cáo	07/04/2026

Bản báo cáo này được biên soạn trực tiếp từ mã nguồn hiện có của hệ thống, bám vào cấu trúc thư mục, tuyến xử lý request, schema dữ liệu, route, controller, service, repository, database adapter và các thành phần tích hợp bên ngoài.

MỤC LỤC

- 1. I. Tổng quan và giới thiệu
- 2. II. Đặc tả và mô hình hóa yêu cầu
- 3. III. Cấu trúc và hành vi
- 4. IV. Kiến trúc và mẫu thiết kế
- 5. V. Minh chứng phần mềm
- 6. Phụ lục A. Danh mục endpoint chính
- 7. Phụ lục B. Tổng hợp thực thể dữ liệu
- 8. Phụ lục C. Ma trận bảng chứng kiến trúc và pattern

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Giải nghĩa
API	Application Programming Interface
RBAC	Role-Based Access Control
CRUD	Create - Read - Update - Delete
JWT	JSON Web Token
OTP	One-Time Password
UML	Unified Modeling Language
C4	Context - Container - Component - Code
NFR	Non-functional Requirement
DTO	Data Transfer Object

I. TỔNG QUAN VÀ GIỚI THIỆU

1. Bối cảnh bài toán

Base Backend API là một hệ thống backend dùng cho bài toán quản lý nội bộ có nhiều nghiệp vụ kết hợp với nhau: xác thực và phân quyền, quản lý hồ sơ thành viên, lập lịch trực, thông báo hệ thống, quản lý thưởng - phạt, quản lý tài nguyên file và xuất báo cáo tổng hợp. Đây không phải là một API đơn chức năng, mà là một nền tảng backend mang tính tích hợp, phục vụ nhiều nhóm tác vụ quản trị trong cùng một miền nghiệp vụ.

Trong môi trường vận hành nội bộ, các hệ thống dạng này thường gặp ba vấn đề lớn. Thứ nhất là dữ liệu bị phân tán, nhiều nghiệp vụ tách rời và khó đồng bộ. Thứ hai là logic phân quyền không nhất quán giữa các nhóm chức năng. Thứ ba là việc phát triển nhanh nhiều chức năng khiến hệ thống dễ rơi vào tình trạng controller quá tải, business logic lẫn với data access, từ đó gây khó khăn cho bảo trì. Codebase đang khảo sát được xây dựng để giải quyết các vấn đề đó bằng cách tổ chức thống nhất theo module nghiệp vụ nhưng vẫn duy trì một backend API chung.

Từ góc nhìn sản phẩm, hệ thống hướng tới nhóm người dùng là thành viên của một tổ chức hoặc đơn vị có nhu cầu quản lý người dùng, lịch trực, truyền thông nội bộ và theo dõi hoạt động. Điều này giải thích vì sao trong cùng một hệ thống lại có sự hiện diện của các module như auth, users, duty, notifications, files, reward-penalties và reports. Mỗi module phục vụ một nhóm nhu cầu rõ ràng, nhưng dữ liệu và quyền hạn lại có liên hệ chặt chẽ với nhau.

Báo cáo này có mục tiêu kép. Một mặt, tài liệu hóa đầy đủ yêu cầu và mô hình hóa phần mềm theo đúng tinh thần môn Kiến trúc và Thiết kế phần mềm. Mặt khác, báo cáo đóng vai trò là một tài liệu kỹ thuật có thể dùng lại để thuyết trình, phản biện hoặc làm cơ sở chỉnh sửa tài liệu cuối cùng của nhóm. Vì vậy, nội dung được triển khai theo hướng vừa học thuật, vừa bám sát mã nguồn đang tồn tại.

2. Giới thiệu tổng quan về sản phẩm

Sản phẩm được xây dựng bằng Node.js kết hợp Express và TypeScript. Toàn bộ hệ thống được khởi động từ một endpoint duy nhất, khởi tạo middleware, mount route và kết nối cơ sở dữ liệu trước khi lắng nghe cổng dịch vụ. Bên trong, mã nguồn được chia theo module nghiệp vụ, mỗi module có route, controller, service và repository riêng. Mô hình tổ chức này giúp hệ thống đủ đơn giản để vận hành như một backend thống nhất, đồng thời đủ rõ ràng để tách trách nhiệm giữa các nhóm chức năng.

Về chức năng, hệ thống có thể hỗ trợ trọn vẹn vòng đời làm việc của người dùng trong môi trường nội bộ. Người dùng có thể đăng ký, đăng nhập, lấy thông tin bản thân, cập nhật hồ sơ, thay đổi mật khẩu, nhận OTP khi quên mật khẩu, xem lịch trực theo tuần, đăng ký ca trực, hủy ca trực và gửi yêu cầu đổi ca. Song song, quản trị viên có thể quản lý tài khoản, thay đổi trạng thái, theo dõi thống kê, duyệt đổi ca, thao tác với dữ liệu import/export và lấy báo cáo tổng quan ở cấp hệ thống.

Một điểm đáng chú ý là hệ thống không chỉ xử lý dữ liệu nghiệp vụ mà còn tích hợp dịch vụ ngoài. Clouinary được dùng cho upload và quản lý tài nguyên media; email/SMS gateway

được dùng để phát OTP khi reset mật khẩu; Swagger UI được dùng để công bố tài liệu API. Điều này cho thấy sản phẩm mang tính thực chiến, không chỉ dừng ở mức CRUD cơ bản trên cơ sở dữ liệu.

3. Mục đích của sản phẩm

- Tập trung hóa các nghiệp vụ quản trị nội bộ trong một backend API thống nhất.
- Chuẩn hóa xác thực, phân quyền và quy tắc truy cập tài nguyên theo role/permission.
- Hỗ trợ quản lý vòng đời người dùng từ đăng ký, kích hoạt, cập nhật hồ sơ đến vô hiệu hóa tài khoản.
- Hỗ trợ vận hành lịch trực, đăng ký ca, đổi ca và thông báo cho các actor liên quan.
- Tạo một nền tảng backend có thể dùng làm base project cho các hệ thống cùng miền nghiệp vụ.

4. Phạm vi và giả định khảo sát

Nội dung	Mô tả
Phạm vi khảo sát	Mã nguồn backend trong repository TCNS-DCD-BACK, tập trung vào route, controller, service, repository, schema và tầng database adapter.
Ngoài phạm vi	Frontend thực tế, cấu hình hạ tầng production, monitoring ngoài ứng dụng và các hệ thống client cụ thể.
Giả định chính	Hệ thống được vận hành như một backend API nội bộ, người dùng tương tác qua frontend hoặc công cụ API client.
Mục tiêu tài liệu	Vừa phục vụ báo cáo học phần vừa có thể dùng như tài liệu kỹ thuật cho chỉnh sửa tiếp theo.

5. Stakeholder và actor

Nhóm liên quan	Vai trò	Kỳ vọng chính
Guest	Người chưa đăng nhập	Có thể đăng ký, đăng nhập và khôi phục mật khẩu nhanh, an toàn
User	Người dùng đã xác thực	Quản lý hồ sơ cá nhân, lịch trực, thông báo và thao tác tệp theo quyền được cấp
Admin	Người quản trị hoặc nhân sự quyền cao	Quản lý người dùng, duyệt nghiệp vụ, báo cáo và tài nguyên hệ thống
Developer	Nhóm phát triển backend	Codebase dễ bảo trì, module rõ ràng, có thể mở rộng và kiểm thử
System owner	Người sở hữu sản phẩm	Hệ thống ổn định, dễ triển khai, hỗ trợ đầy đủ yêu cầu vận hành

6. Các module nghiệp vụ chính

Module	Mục đích	Ví dụ chức năng
auth	Xác thực và bảo mật	Đăng ký, đăng nhập, đổi mật

		khẩu, refresh token, quên mật khẩu bằng OTP
users	Quản lý hồ sơ người dùng	CRUD user, cập nhật profile, quản lý trạng thái, import/export, thống kê
duty	Quản lý lịch trực	Tạo ca trực, đăng ký ca, hủy ca, đổi ca, thống kê lịch trực
notifications	Thông báo nội bộ	Lấy danh sách thông báo, đánh dấu đã đọc, cài đặt cấu hình nhận thông báo
files	Upload và quản lý tài nguyên	Upload avatar, upload file, xem thông tin file, xóa file, thống kê storage
reward-penalties	Ghi nhận thưởng - phạt	Tạo bản ghi thưởng/phạt, xem lịch sử, xem thống kê tài chính
reports	Báo cáo tổng quan	Tổng hợp dữ liệu người dùng, trực, tài chính, thông báo và export báo cáo

7. Yêu cầu chức năng chi tiết

Mã	Yêu cầu chức năng	Mức ưu tiên	Actor
FR01	Đăng ký tài khoản người dùng mới với email và mật khẩu.	Cao	Guest
FR02	Đăng nhập và nhận access token cùng refresh token.	Cao	Guest
FR03	Lấy thông tin người dùng hiện tại qua endpoint me.	Cao	User
FR04	Đổi mật khẩu khi đã đăng nhập.	Cao	User
FR05	Yêu cầu OTP để quên mật khẩu và đặt lại mật khẩu.	Cao	Guest / User
FR06	Quản trị viên tạo user mới từ giao diện quản trị hoặc API.	Trung bình	Admin
FR07	Quản trị viên cập nhật, vô hiệu hóa hoặc xóa người dùng.	Cao	Admin
FR08	Người dùng tự cập nhật hồ sơ cá nhân và avatar.	Cao	User
FR09	Hệ thống cho phép import/export dữ liệu người dùng bằng CSV/XLSX.	Trung bình	Admin
FR10	Người dùng xem lịch	Cao	User / Admin

	trực tuần hiện tại hoặc tuần được chỉ định.		
FR11	Quản trị viên tạo và cập nhật ca trực.	Cao	Admin
FR12	Người dùng đăng ký ca trực nếu còn chỗ và không xung đột thời gian.	Cao	User
FR13	Người dùng hủy đăng ký ca trực của chính mình.	Cao	User
FR14	Người dùng tạo yêu cầu đổi ca với người dùng đích.	Cao	User
FR15	Quản trị viên duyệt hoặc từ chối yêu cầu đổi ca.	Cao	Admin
FR16	Người dùng xem, đánh dấu đã đọc, xóa thông báo và thay đổi cấu hình nhận thông báo.	Cao	User
FR17	Quản trị viên tạo bản ghi thưởng - phạt và xem thống kê tài chính.	Trung bình	Admin
FR18	Người dùng upload avatar hoặc file dùng chung; quản trị viên xem thông tin và xóa file.	Cao	User / Admin
FR19	Quản trị viên xem báo cáo tổng quan và export CSV/XLSX.	Trung bình	Admin
FR20	Hệ thống cung cấp tài liệu API và endpoint health check.	Trung bình	Admin / Developer

8. Yêu cầu phi chức năng

Nhóm NFR	Nội dung chi tiết
Bảo mật	Mật khẩu phải được băm; endpoint nhạy cảm cần xác thực JWT; phân quyền dựa trên permission; giới hạn rate cho login/register ở môi trường production.
Tính đúng đắn dữ liệu	Schema validation cần kiểm tra ràng buộc kiểu dữ liệu, unique field, foreign key và custom validation cho password.
Tính sẵn sàng	Ứng dụng phải có health check, xử lý lỗi tập trung và khởi tạo database trước khi nhận request.
Hiệu năng	Các endpoint danh sách cần hỗ trợ phân trang, lọc, sort và query normalization để giảm tải cho client.

Khả năng mở rộng	Code cần cho phép thêm module mới theo cùng cấu trúc routes/controllers/services/repositories mà không phải sửa quá nhiều thành phần lỗi.
Khả năng bảo trì	Tổ chức module rõ ràng, có shared/common, base service và base repository để giảm lặp nhưng không làm mờ trách nhiệm.
Khả năng quan sát	Có request logging, mã lỗi nhất quán và tài liệu Swagger để hỗ trợ debug và trao đổi giữa nhóm phát triển.
Khả năng tích hợp	Hệ thống phải giao tiếp được với Cloudinary, email/SMS gateway và các công cụ client qua giao thức HTTP chuẩn.

9. Ràng buộc và giả định kỹ thuật

- Hệ thống đang vận hành như một backend API duy nhất, không giả định có microservice tách riêng.
- Runtime hiện tại khởi tạo MongoDB thông qua database adapter, dù tài liệu môi trường có dấu vết cấu hình DB_CONNECTION cũ.
- Một số luồng như OTP phụ thuộc vào gateway ngoài; trong môi trường development có thể mock nếu chưa cấu hình URL thực.
- Phần minh chứng UI không có sẵn trong repository nên báo cáo chỉ chứa khung ảnh demo để người dùng tự chèn sau.

II. ĐẶC TẢ VÀ MÔ HÌNH HÓA YÊU CẦU

1. Cách tiếp cận mô hình hóa

Phần này mô hình hóa yêu cầu chức năng bằng UML ở mức đủ dùng cho bài tập lớn. Do tài liệu được tạo trong định dạng DOCX để dễ chỉnh sửa, các biểu đồ được biểu diễn dưới dạng text UML thay vì ảnh tĩnh. Cách trình bày này giúp người đọc chỉnh tên actor, use case hoặc luồng xử lý trực tiếp trong tài liệu mà không phải dùng phần mềm vẽ ngoài.

Các use case được lựa chọn đều là những nghiệp vụ trung tâm và có dấu vết rõ trong mã nguồn: đăng nhập, quản lý user, cập nhật profile/avatar, xem lịch trực, đăng ký ca, gửi yêu cầu đổi ca, quản lý thông báo, upload file và xuất báo cáo. Điều này giúp UML không bị chung chung mà phản ánh đúng những gì hệ thống đang hỗ trợ thực tế.

2. Use Case Diagram tổng quát



	Admin	-----> (Quan ly tai nguyen Cloudinary)
+	-----	
-----+		

3. Phân tích actor

Actor	Mô tả	Tài nguyên thường truy cập
Guest	Người chưa xác thực, chưa có thông tin phiên đăng nhập trong hệ thống.	register, login, forgot-password, reset-password
User	Người dùng đã đăng nhập, có id, role và permission trong request context.	me, profile, duty, notifications, upload
Admin	Người dùng có permission cao và có thể quản trị tài khoản, lịch trực, file và báo cáo.	users, reports, reward-penalties, upload management, duty management

4. Đặc tả use case chi tiết

UC01. Đăng nhập hệ thống

Thuộc tính	Nội dung
Mã use case	UC01
Tên use case	Đăng nhập hệ thống
Actor chính	Guest
Mục tiêu	Cho phép người dùng xác thực và nhận token truy cập hệ thống.
Tiền điều kiện	Người dùng có tài khoản hợp lệ đã được tạo trong hệ thống.
Hậu điều kiện	Người dùng đăng nhập thành công và có thể gọi các endpoint cần xác thực.

Luồng chính:

- Người dùng nhập email và mật khẩu.
- Client gửi POST /api/auth/login.
- Route kiểm tra các trường bắt buộc email và password.
- Controller chuyển payload cho AuthService.
- AuthService chuẩn hóa email và tìm user theo email.
- Hệ thống kiểm tra trạng thái active của tài khoản.
- Hệ thống so sánh mật khẩu đầu vào với mật khẩu đã băm.
- Nếu hợp lệ, hệ thống cập nhật lastLogin.
- Hệ thống sinh access token và refresh token.
- Controller trả response chứa thông tin user đã sanitize, quyền và token.

Luồng thay thế / ngoại lệ:

- Nếu email không tồn tại, hệ thống trả lỗi 401.

- Nếu tài khoản bị vô hiệu hóa, hệ thống trả lỗi 401.
- Nếu mật khẩu sai, hệ thống trả lỗi 401 và không sinh token.

UC02. Quản lý người dùng

Thuộc tính	Nội dung
Mã use case	UC02
Tên use case	Quản lý người dùng
Actor chính	Admin
Mục tiêu	Tạo mới, cập nhật, thay đổi trạng thái hoặc xóa người dùng.
Tiền điều kiện	Admin đã đăng nhập và có permission phù hợp như users:create, users:update, users:delete.
Hậu điều kiện	Trạng thái và dữ liệu người dùng được cập nhật đúng theo action quản trị.

Luồng chính:

- Admin truy cập endpoint quản lý người dùng.
- Route kiểm tra xác thực và permission.
- Controller đọc request và gọi UserService hoặc service liên quan.
- Service kiểm tra ràng buộc dữ liệu, chuẩn hóa payload, băm mật khẩu nếu cần.
- Repository ghi dữ liệu vào collection users.
- Controller trả về thông tin user sau khi sanitize.

Luồng thay thế / ngoại lệ:

- Nếu dữ liệu vi phạm schema hoặc unique field, hệ thống trả lỗi validation.
- Nếu admin cố thao tác hủy tài khoản của chính mình trong các luồng bị cấm, hệ thống ném ApiError tương ứng.
- Nếu user không tồn tại, service trả thông báo not found.

UC03. Cập nhật hồ sơ và avatar

Thuộc tính	Nội dung
Mã use case	UC03
Tên use case	Cập nhật hồ sơ và avatar
Actor chính	User
Mục tiêu	Cho phép người dùng cập nhật thông tin cá nhân và thay đổi avatar.
Tiền điều kiện	Người dùng đã đăng nhập.
Hậu điều kiện	Hồ sơ người dùng và avatar được cập nhật nhất quán giữa DB và storage.

Luồng chính:

- Người dùng gửi request PUT /api/users/profile kèm dữ liệu hồ sơ và có thể kèm file avatar.
- Route kiểm tra xác thực và gắn middleware upload.
- Controller lọc ra các trường được phép cập nhật.
- Nếu có file, UserAvatarService xử lý upload avatar.

29. UploadService chuẩn bị file, upload lên Cloudinary và lưu metadata file record.

30. Service cập nhật user với avatar mới và các trường hồ sơ hợp lệ.

31. Controller trả response user đã sanitize.

Luồng thay thế / ngoại lệ:

- Nếu request không có trường nào hợp lệ và cũng không có file, hệ thống trả lỗi bad request.
- Nếu upload thành công nhưng ghi DB file record thất bại, service rollback xóa file đã upload trên Cloudinary.

UC04. Đăng ký ca trực

Thuộc tính	Nội dung
Mã use case	UC04
Tên use case	Đăng ký ca trực
Actor chính	User
Mục tiêu	Cho phép người dùng đăng ký vào một duty slot còn chỗ.
Tiền điều kiện	Người dùng đã đăng nhập và có quyền duty:register.
Hậu điều kiện	Người dùng được gán vào duty slot và nhận thông báo xác nhận.

Luồng chính:

32. Người dùng chọn một ca trực.

33. Client gửi PATCH /api/duty/slots/:id/register.

34. Route kiểm tra requireAuth và requirePermission.

35. DutyService tìm slot theo id.

36. Service kiểm tra slot có bị locked hay không.

37. Service kiểm tra user đã đăng ký trước đó chưa.

38. Service kiểm tra capacity và xung đột thời gian với các slot khác.

39. Nếu hợp lệ, service cập nhật assignedUserIds.

40. Service phát thông báo xác nhận đăng ký ca cho user.

41. Controller trả slot hoặc kết quả cập nhật.

Luồng thay thế / ngoại lệ:

- Nếu slot đã khóa, trả lỗi không thể đăng ký.
- Nếu slot đầy, trả lỗi duty slot is full.
- Nếu trùng khung giờ với ca khác đã nhận, trả lỗi xung đột lịch.

UC05. Gửi yêu cầu đổi ca

Thuộc tính	Nội dung
Mã use case	UC05
Tên use case	Gửi yêu cầu đổi ca
Actor chính	User
Mục tiêu	Cho phép người dùng đề xuất chuyển ca trực cho một người dùng khác và chờ duyệt.
Tiền điều kiện	Người dùng đang sở hữu ca trực và có quyền

	duty:update.
Hậu điều kiện	Yêu cầu đổi ca được lưu lại và chờ quản trị viên phê duyệt hoặc từ chối.

Luồng chính:

42. Người dùng chọn ca trực đã đăng ký và nhập người nhận ca cùng lý do.
43. Client gửi POST /api/duty/swaps.
44. DutyService kiểm tra slot tồn tại, requester là người đang giữ ca và target hợp lệ.
45. Service tạo bản ghi duty swap request với trạng thái pending.
46. Service phát thông báo cho requester, target user và nhóm approver.
47. Controller trả bản ghi yêu cầu đổi ca vừa tạo.

Luồng thay thế / ngoại lệ:

- Nếu requester không giữ ca hoặc target không hợp lệ, service trả lỗi bad request.
- Nếu slot/target user không tồn tại, hệ thống trả lỗi not found tương ứng.

UC06. Duyệt yêu cầu đổi ca

Thuộc tính	Nội dung
Mã use case	UC06
Tên use case	Duyệt yêu cầu đổi ca
Actor chính	Admin
Mục tiêu	Cho phép quản trị viên chấp thuận hoặc từ chối yêu cầu đổi ca.
Tiền điều kiện	Admin đã đăng nhập và có quyền duty:approve_swap.
Hậu điều kiện	Swap request được cập nhật trạng thái approved hoặc rejected và hệ thống đồng bộ dữ liệu ca trực tương ứng.

Luồng chính:

48. Admin xem danh sách yêu cầu đổi ca pending.
49. Admin gửi quyết định approve hoặc reject qua PATCH /api/duty/swaps/:id/decision.
50. DutyService phân tích payload decision và decisionNote.
51. Nếu duyệt, service cập nhật assignedUserIds của slot và đổi trạng thái swap request.
52. Service phát thông báo cho requester và target user.
53. Controller trả kết quả quyết định.

Luồng thay thế / ngoại lệ:

- Nếu decision không hợp lệ, service trả lỗi bad request.
- Nếu dữ liệu slot thay đổi khiến requester không còn sở hữu ca, service báo lỗi và không apply swap.

UC07. Quản lý thông báo

Thuộc tính	Nội dung
Mã use case	UC07
Tên use case	Quản lý thông báo

Actor chính	User
Mục tiêu	Cho phép người dùng xem, đánh dấu đã đọc, xóa thông báo và thay đổi cấu hình nhận thông báo.
Tiền điều kiện	Người dùng đã đăng nhập.
Hậu điều kiện	Danh sách thông báo và cấu hình nhận thông báo của user được cập nhật phù hợp.

Luồng chính:

54. Người dùng truy cập /api/notifications để lấy danh sách thông báo.
55. NotificationService lấy dữ liệu theo userId, có hỗ trợ phân trang và đếm unread.
56. Người dùng có thể gọi endpoint mark as read cho một notification.
57. Người dùng có thể gọi endpoint read-all hoặc delete.
58. Người dùng có thể cập nhật settings như shiftNotifications, approvalNotifications, emailNotifications, smsNotifications.

Luồng thay thế / ngoại lệ:

- Nếu notification không thuộc về user hiện tại, service trả lỗi not found.
- Nếu payload settings không có khóa hợp lệ, service giữ nguyên settings cũ.

UC08. Upload file và quản lý tài nguyên

Thuộc tính	Nội dung
Mã use case	UC08
Tên use case	Upload file và quản lý tài nguyên
Actor chính	User / Admin
Mục tiêu	Cho phép upload avatar hoặc file chung, đồng thời cho admin quản lý tài nguyên lưu trữ.
Tiền điều kiện	Người dùng phải xác thực; một số endpoint quản lý file yêu cầu role admin.
Hậu điều kiện	Tài nguyên file được lưu nhất quán giữa Cloudinary và metadata store.

Luồng chính:

59. User gửi file đến endpoint upload tương ứng.
60. Middleware upload policy kiểm tra field name, loại file và giới hạn.
61. UploadService xử lý ảnh hoặc file chung, gọi CloudinaryStorageService để upload.
62. FileService lưu metadata file record vào cơ sở dữ liệu.
63. Admin có thể gọi endpoint lấy info file, xóa file hoặc dọn file cũ.
64. Hệ thống trả thông tin tài nguyên trên Cloudinary và metadata liên quan.

Luồng thay thế / ngoại lệ:

- Nếu Cloudinary chưa cấu hình, service trả lỗi cấu hình storage.
- Nếu ghi DB thất bại sau khi upload, hệ thống rollback xóa file đã upload.
- Nếu file không tồn tại, endpoint info/delete trả not found.

5. Activity Diagram - Quy trình đăng nhập

```

Start
|
Nhap email + password
|
Validate request body
|
Chuan hoa email
|
Tim user theo email
|
User ton tai?
|-- No --> Tra loi 401 -> End
|
User active?
|-- No --> Tra loi 401 -> End
|
Compare password
|
Password dung?
|-- No --> Tra loi 401 -> End
|
Cap nhat lastLogin
|
Sinh token + refresh token
|
Tra response JSON
|
End

```

Activity trên cho thấy luồng đăng nhập là một chuỗi quyết định tuyến tính với nhiều điểm fail-fast. Điều này phù hợp với yêu cầu bảo mật, vì hệ thống ngắt sớm khi thông tin xác thực không hợp lệ thay vì đi tiếp đến các bước phát token.

6. Activity Diagram - Quy trình gửi và duyệt đổi ca

```

Start
|
User chọn duty slot và nhập target user + reason
|
Kiểm tra quyền duty:update
|
Kiểm tra slot tồn tại và requester đang giữ ca
|
Kiểm tra target user hợp lệ
|
Tạo swap request (pending)
|
Gửi thông báo requester / target / approver
|
Cho quyết định admin
|
Admin approve?
|-- No --> Cập nhật status rejected + gửi thông báo -> End
|
Cập nhật assignedUserIds trong duty slot
|
Cập nhật status approved + gửi thông báo
|
End

```

Luồng dữ liệu thể hiện một nghiệp vụ có tương tác nhiều bên: requester, target user và approver. Đây là ví dụ tốt để minh họa việc service layer điều phối nhiều repository và nhiều thông báo thay vì xử lý trực tiếp trong controller.

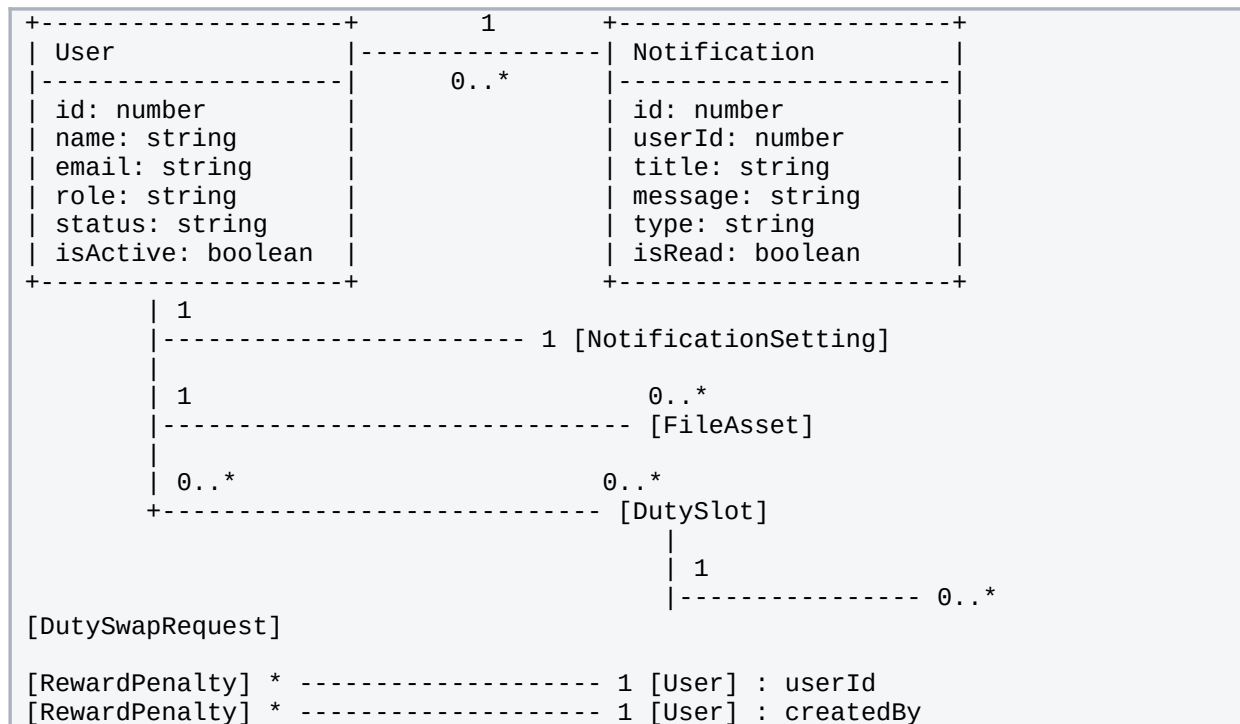
7. Activity Diagram - Quy trình upload avatar

```
graph TD
    Start([Start]) --> UserRequest[User gui request PUT /users/profile kèm file avatar]
    UserRequest --> Middleware[Middleware kiểm tra auth + file policy]
    Middleware --> Controller[Controller loc truong duoc cap nhat]
    Controller --> ProcessFile[Xu ly file (prepare avatar)]
    ProcessFile --> Upload[Upload len Cloudinary]
    Upload --> SaveDB[Luu file record vao DB]
    SaveDB --> UpdateAvatar[Cap nhat user.avatar]
    UpdateAvatar --> CheckDB[Loi khi ghi DB sau upload?]
    CheckDB -- Yes --> Rollback[Rollback xoa file tren Cloudinary -> End]
    CheckDB -- No --> Sanitize[Tra response user da sanitize]
    Sanitize --> End([End])
```

Luồng upload avatar cho thấy hệ thống đã tính đến tính nhất quán dữ liệu khi làm việc với một dịch vụ ngoài. Đây là một điểm mạnh quan trọng vì nhiều hệ thống upload file chỉ upload thành công nhưng không xử lý trường hợp ghi metadata nội bộ thất bại.

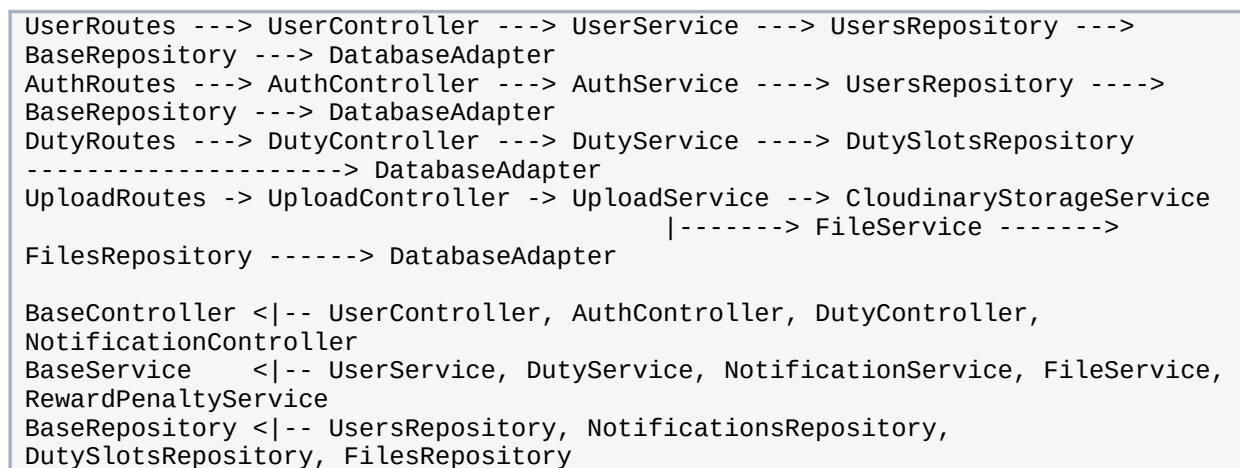
III. CẤU TRÚC VÀ HÀNH VI

1. Class Diagram miền dữ liệu



Class diagram miền dữ liệu tập trung vào các thực thể phản ánh dữ liệu nghiệp vụ của hệ thống. Đây là lớp mô hình hóa gần với schema hơn là với source code lớp thực thi. Cách nhìn này hữu ích khi phân tích quan hệ nghiệp vụ và xác định phạm vi tác động khi thay đổi dữ liệu.

2. Class Diagram tầng ứng dụng



Sơ đồ tầng ứng dụng cho thấy đây là một hệ thống phân tầng theo đúng nghĩa: route chỉ định nghĩa endpoint và middleware; controller chỉ giao tiếp HTTP; service chứa nghiệp vụ; repository truy cập dữ liệu; DatabaseAdapter che giấu chi tiết của persistence. Việc dùng **BaseController**, **BaseService** và **BaseRepository** giúp giảm lặp code nhưng không thay đổi trách nhiệm cốt lõi của từng layer.

3. Mô tả các lớp/thành phần trọng tâm

Thành phần	Vai trò	Nhận xét
AuthController	Nhận request auth, validate đầu vào, gọi AuthService, trả response	Controller mỏng, không tự làm nghiệp vụ xác thực
AuthService	Xử lý register, login, refresh token, change password, forgot/reset password	Là nơi tập trung logic xác thực và quyền
UserController	Nhận request quản lý user, profile, avatar, stats	Chủ yếu điều phối sang UserService hoặc UserAvatarService
UserService	Chứa logic nghiệp vụ về người dùng, thống kê, chuẩn hóa dữ liệu, tác động chéo sang module khác	Là service khá lớn và giàu nghiệp vụ
DutyService	Quản lý duty slot, đăng ký ca, hủy ca, đổi ca, thống kê duty	Điều phối nhiều rule nghiệp vụ và thông báo
NotificationService	Gửi thông báo, lấy thông báo, quản lý settings	Gắn chặt với nhiều flow nghiệp vụ khác
UploadService	Điều phối upload, xử lý file, lưu metadata, rollback	Đóng vai trò facade cho luồng upload
BaseService	Cung cấp CRUD chung, import/export, validation theo schema và hook before/after	Thể hiện Template Method rõ rệt
BaseRepository	Chuẩn hóa data access	Giúp service không gọi trực tiếp database
DatabaseAdapter / MongoConnect	Abstraction và implementation persistence	Che giấu chi tiết của Mongoose/MongoDB

4. Sequence Diagram - Luồng đăng nhập

```
Guest -> AuthRoute      : POST /api/auth/login
AuthRoute -> Validator   : validateFields(email, password)
Validator --> AuthRoute  : hop le
AuthRoute -> AuthController : login(req, res)
AuthController -> AuthService : login(payload)
AuthService -> UsersRepository : findByEmail(email)
UsersRepository -> BaseRepository : findOne(query)
BaseRepository -> DatabaseAdapter : findOne('users', query)
DatabaseAdapter --> BaseRepository : user
BaseRepository --> UsersRepository : user
UsersRepository --> AuthService : user
AuthService -> AuthService : comparePassword(...)
AuthService -> UsersRepository : updateLastLogin(user.id, now)
AuthService -> JWT helper : generateToken(), generateRefreshToken()
JWT helper --> AuthService : token + refreshToken
AuthService --> AuthController : authResponse
AuthController --> Guest : 200 OK + user + permissions + token
```

5. Sequence Diagram - Luồng upload avatar

```
User -> UserRoute      : PUT /api/users/profile + file
```

```

UserRoute -> AuthMiddleware : requireAuth
UserRoute -> UploadMiddleware: parse avatar/image field
UserRoute -> UserController : updateProfile(req, res)
UserController -> UserAvatarService : updateUserWithAvatar(...)
UserAvatarService -> UploadService : uploadAvatar(file, userId)
UploadService -> FileProcessingService : prepareAvatar(file)
UploadService -> CloudinaryStorageService : upload(buffer, folder, name)
CloudinaryStorageService --> UploadService : cloudinary asset
UploadService -> FileService : saveFileRecord(asset, metadata)
FileService -> FilesRepository : create(record)
FilesRepository -> DatabaseAdapter : create('files', record)
UploadService --> UserAvatarService : asset + fileRecord
UserAvatarService -> UserService/UsersRepository : update user.avatar
UserAvatarService --> UserController : updated user
UserController --> User : 200 OK + sanitized user

```

6. Sequence Diagram - Luồng duyệt đổi ca

```

Admin -> DutyRoute : PATCH /api/duty/swaps/:id/decision
DutyRoute -> AuthMiddleware: requireAuth
DutyRoute -> RBAC : requirePermission('duty:approve_swap')
DutyRoute -> DutyController: decideSwap(req, res)
DutyController -> DutyService: decideSwap(id, payload, admin)
DutyService -> DutySwapRequestsRepository : findById(id)
DutyService -> DutyService : parse decision
DutyService -> DutySlotsRepository : findById(dutySlotId)
DutyService -> DutyService : applyApprovedSwapRequest() neu approve
DutyService -> DutySlotsRepository : update assignedUserIds
DutyService -> DutySwapRequestsRepository : update status + approvedBy + approvedAt
DutyService -> NotificationService : notify requester / target
DutyService --> DutyController : result
DutyController --> Admin : 200 OK

```

7. Phân tích cấu trúc và hành vi

Ba sequence diagram trên thể hiện ba kiểu luồng hành vi tiêu biểu của hệ thống. Luồng đăng nhập là một luồng xác thực tuyến tính, tập trung vào kiểm tra tính hợp lệ và sinh token. Luồng upload avatar là luồng tích hợp ngoài, trong đó nhất quán dữ liệu giữa Cloudinary và database là trọng tâm. Luồng duyệt đổi ca là luồng nghiệp vụ phối hợp nhiều repository và nhiều thông báo, đại diện cho những use case phức hợp hơn mức CRUD thông thường.

Từ góc nhìn thiết kế logic, hệ thống có sự phân tách khá tốt giữa phần 'orchestration' và phần 'data manipulation'. Các service như DutyService hay UploadService làm nhiệm vụ điều phối. Các repository chỉ lo query/ghi dữ liệu. Cách tách này khiến việc mô tả sequence diagram trở nên mạch lạc vì trách nhiệm của mỗi participant khá rõ. Đây là một dấu hiệu tích cực cho chất lượng thiết kế nội bộ của codebase.

Một điểm nữa đáng lưu ý là các thực thể dữ liệu có nhiều quan hệ chéo xoay quanh User. Điều này làm User trở thành hạt nhân của miền dữ liệu. Trong tương lai, nếu cần refactor hoặc mở rộng hệ thống, mọi thay đổi về user lifecycle, permission hoặc status đều có khả năng lan tỏa sang các module notification, duty, files, reward-penalty và reports. Việc nhận diện mối liên kết đó ngay từ giai đoạn class diagram rất quan trọng đối với công tác bảo trì.

IV. KIẾN TRÚC VÀ MẪU THIẾT KẾ

1. Bảng chứng nhận diện kiến trúc hệ thống

Bảng chứng từ code	Ý nghĩa kiến trúc
index.ts chỉ import dotenv và src/server rồi khởi động ứng dụng	Hệ thống có một entrypoint runtime duy nhất
src/server.ts khởi tạo một Express app, mount middleware, mount routes và listen một cổng	Tất cả nghiệp vụ chạy trong một process ứng dụng thống nhất
src/routes/index.ts mount các module auth, users, upload, files, notifications, duty, reward-penalties, reports	Các module là boundary logic trong cùng một backend, không phải service deploy riêng
Service như UserService import trực tiếp repository và service từ module khác	Giao tiếp nội bộ diễn ra bằng gọi hàm trực tiếp, không qua service-to-service network
BaseRepository dùng chung DatabaseAdapter cho các module	Tầng persistence thống nhất, không có data ownership tách theo microservice

Tập hợp bằng chứng trên dẫn tới kết luận rằng hệ thống hiện tại là Monolith, cụ thể hơn là Modular Monolith. Các module nghiệp vụ được chia rõ trong mã nguồn nhưng không tách rời về runtime, deployment hay quyền sở hữu dữ liệu.

2. So sánh Monolith, SOA và Microservice

Tiêu chí	Monolith	SOA	Microservice	Kết quả đối với project
Số lượng ứng dụng deploy	Một	Nhiều service lớn	Nhiều service nhỏ	Một ứng dụng Express duy nhất
Giao tiếp nội bộ	Gọi hàm trực tiếp	HTTP / bus / broker	HTTP / gRPC / queue	Import service/repository trực tiếp
Quyền sở hữu dữ liệu	Dùng chung hoặc tập trung	Có thể tách	Ưu tiên tách riêng	Dùng chung database adapter
Độ phức tạp vận hành	Thấp hơn	Trung bình	Cao	Phù hợp với hiện trạng codebase
Tính độc lập phát hành	Thấp	Trung bình	Cao	Không có bằng chứng deploy độc lập từng module

Nếu hệ thống là SOA hoặc microservice, báo cáo phải phát hiện ít nhất một trong các dấu hiệu sau: nhiều service package chạy độc lập, nhiều entrypoint, API gateway hoặc service mesh nội bộ, giao tiếp HTTP/gRPC/queue giữa service, hoặc data store được chia theo từng service. Codebase hiện tại không cho thấy các dấu hiệu này.

Ngược lại, project bộc lộ đầy đủ đặc trưng của modular monolith: cấu trúc module rõ ràng, dependency nội bộ trực tiếp, runtime thống nhất, nhưng vẫn có nhiều boundary logic để giảm độ phức tạp nhận thức. Đây là một lựa chọn phù hợp cho hệ thống có nhiều nghiệp vụ nội bộ nhưng chưa cần gánh chi phí vận hành của kiến trúc phân tán.

3. Mẫu kiến trúc đang áp dụng

Mẫu kiến trúc	Có áp dụng hay không	Phân tích
N-layered / Modular Layered	Có	Luồng Route -> Controller -> Service -> Repository -> Database Adapter xuất hiện rõ ở hầu hết module
MVC	Một phần	Có controller và model dữ liệu nhưng không có view nội bộ; hệ thống thiên về REST API hơn là web MVC hoàn chỉnh
CQRS	Không	Không tách command model, query model, handler riêng hay projection store riêng
Clean Architecture	Chưa đầy đủ	Có vài abstraction và DI nhẹ nhưng chưa có dependency rule chặt và chưa tách rõ domain/application/infrastructure

```
Request
-> Route
-> Middleware auth / validation / query
-> Controller
-> Service
-> Repository
-> DatabaseAdapter
-> MongoDB
-> Response
```

Mẫu kiến trúc phù hợp nhất là Modular Layered Architecture. Mỗi module là một khối nghiệp vụ, còn bên trong mỗi module lại chia theo tầng trách nhiệm. Cách tổ chức này phù hợp với backend CRUD có thêm business rule, permission, import/export và một số tích hợp ngoài.

4. Vì sao hệ thống chưa phải CQRS

CQRS yêu cầu ít nhất một mức tách biệt rõ ràng giữa phần ghi (command) và phần đọc (query). Sự tách này có thể thể hiện qua handler riêng, model riêng, service riêng hoặc storage tối ưu khác nhau. Trong codebase hiện tại, các thao tác đọc và ghi đều đi chung qua service/repository. BaseService thậm chí cung cấp đồng thời findAll, findById, create, update, delete, search, bulk operations và import/export.

Middleware parseApiQuery chỉ làm nhiệm vụ chuẩn hóa query string thành cấu trúc filter/page/limit/sort/order cho repository. Đây không phải là bằng chứng của CQRS mà chỉ là hỗ trợ query nhất quán. Không có query handler riêng, command handler riêng, event sourcing hay projection read model. Vì vậy, xếp hệ thống này vào CQRS sẽ làm sai bản chất thiết kế hiện tại.

5. Vì sao hệ thống chưa phải Clean Architecture đầy đủ

Clean Architecture yêu cầu vòng phụ thuộc hướng vào trong, tách entities, use cases, interface adapters và frameworks/infrastructure thành các lớp rõ ràng. Trong khi đó, hệ thống hiện tại

vẫn theo phong cách practical layered architecture. Service import concrete repository; controller import concrete service singleton; repository import concrete db proxy. Điều này cho thấy dependency vẫn được tổ chức theo phân tầng thực dụng hơn là theo dependency rule nghiêm ngặt của Clean Architecture.

Dù vậy, project có một số hạt giống tích cực. BaseRepository phụ thuộc vào interface DatabaseAdapter; một số service nhận dependency qua constructor mặc định; database được khởi tạo tập trung; nhiều quy tắc validation được gom vào service/schema thay vì rải rác. Các yếu tố này cho thấy codebase có thể tiến gần hơn tới kiến trúc sạch nếu nhu cầu bảo trì hoặc testability tăng cao trong tương lai. Nhưng tại thời điểm hiện tại, kết luận đúng hơn vẫn là 'chưa phải clean architecture đầy đủ'.

6. Phân tích design pattern theo nhóm

6.1. Mẫu tạo dựng

Pattern / kỹ thuật	Vị trí	Lợi ích	Giới hạn
Singleton-style module instance	Nhiều file `export default new ...` ở controller/service/repository	Dùng chung instance, ít boilerplate khởi tạo	Không linh hoạt như container thật sự nếu hệ thống cần thay dependency động
Constructor injection nhẹ	AuthService, UploadService	Cho phép thay dependency khi test hoặc mở rộng	Chưa có DI container, chưa có composition root rõ ràng

Nhóm tạo dựng trong codebase thiên về giải pháp thực dụng hơn là pattern GoF đầy đủ. Tuy nhiên, điều này phù hợp với quy mô hiện tại vì hệ thống vẫn ưu tiên sự đơn giản và khả năng đọc hiểu nhanh của developer.

6.2. Mẫu cấu trúc

Pattern	Áp dụng ở đâu	Giải thích
Repository Pattern	BaseRepository và các repository con	Tách service khỏi chi tiết thao tác dữ liệu và chuẩn hóa API data access
Adapter Pattern	DatabaseAdapter và MongoConnect	Che giấu chi tiết persistence, giúp repository làm việc với một contract chung
Facade-like Pattern	UploadService	Gói nhiều bước xử lý upload, storage, file record và rollback dưới một giao diện đơn giản
Base class reuse	BaseController	Giảm lặp cho CRUD handler và chuẩn hóa cách trả response ở HTTP layer

Repository Pattern là pattern có ảnh hưởng lớn nhất ở codebase, vì nó định hình rõ quan hệ giữa service và database. Service không gọi thẳng Model của Mongoose; thay vào đó, mọi thao tác

dữ liệu đều đi qua repository hoặc base repository. Điều này giúp thay đổi logic query tập trung hơn và tránh trộn lẫn persistence logic vào business layer.

Adapter Pattern xuất hiện ngay dưới repository. DatabaseAdapter tạo ra một biên trừu tượng cho persistence. Dù runtime hiện tại chỉ còn MongoDB, việc có một interface chung vẫn rất có ích vì nó giảm coupling ở tầng repository và giúp giải thích kiến trúc hệ thống rõ ràng hơn trong tài liệu.

6.3. Mẫu hành vi

Pattern	Áp dụng ở đâu	Giá trị thực tế
Template Method	BaseService	Định nghĩa khung create/update/delete với các hook validateCreate, beforeCreate, afterCreate...
Chain of Responsibility	Middleware Express	Cho phép auth, validation, query parsing và error handling chạy theo chuỗi để mở rộng
Service Layer	Hầu hết module services	Tập trung business logic, giữ controller mỏng và repository gọn

Template Method trong BaseService là một ví dụ điển hình: các bước của CRUD được cố định nhưng vẫn cho phép service con chèn logic tùy chỉnh. Đây là cách thiết kế hiệu quả khi nhiều module cùng chia sẻ một khung xử lý tương tự nhưng vẫn có khác biệt nghiệp vụ.

Chain of Responsibility là pattern gần như tự nhiên trong Express. Điểm mạnh của hệ thống là đã tận dụng pattern này khá sạch: route gắn requireAuth, requirePermission, validateFields hoặc validateSchema trước khi chuyển cho controller. Nhờ đó controller ít phải xử lý các nhánh kiểm tra lặp lại.

7. C4 Model - Level 1



Thành phần	Loại	Vai trò trong bối cảnh hệ thống
Người dùng / Frontend	Actor	Gửi request đến backend để thao tác các nghiệp vụ quản lý nội bộ
TCNS Base Backend API	System	Trung tâm xử lý xác thực, hồ sơ, lịch trực, thông báo, upload và báo cáo
MongoDB	External database	Lưu dữ liệu hệ thống thông qua

		Mongo adapter
Cloudinary	External storage service	Lưu file và ảnh đại diện, cung cấp quản lý tài nguyên media
Email/SMS OTP Gateway	External communication service	Gửi OTP trong quy trình quên mật khẩu

8. C4 Model - Level 2

```

Container 1: Express API Monolith
- middleware: auth, rbac, schema validation, api query
- routes: auth, users, duty, notifications, files, reports, reward-penalties
- controllers: HTTP orchestration
- services: business logic
- repositories: data access
- swagger docs
  |
  +--> Container 2: MongoDB
  +--> External: Cloudinary
  +--> External: OTP Gateway

```

Ở Level 2, container trung tâm duy nhất của nội bộ hệ thống là Express API Monolith. Đây là điểm rất quan trọng khi trình bày kiến trúc: routes/controllers/services/repositories không phải container tách biệt, mà là thành phần bên trong cùng một container ứng dụng.

Nếu cần đi tiếp tới Level 3, khi đó mới hợp lý để bóc tách các component như AuthController, AuthService, UsersRepository, UploadService hay DutyService. Tuy nhiên, với phạm vi bài tập lớn hiện tại, Level 1 và Level 2 đã đủ để mô tả bản chất hệ thống và mối quan hệ với các external systems.

9. Đánh giá ưu điểm, hạn chế và định hướng cải tiến

Khía cạnh	Ưu điểm	Hạn chế / rủi ro	Định hướng cải tiến
Tổ chức code	Module hóa rõ, dễ đọc	Service lớn có thể dẫn đến công kênh	Chia nhỏ use case trong cùng module khi cần
Runtime	Triển khai đơn giản	Không có lợi ích deploy độc lập từng phần	Chỉ tách service khi có nhu cầu thật sự
Data access	Repository + adapter rõ ràng	Hiện tại runtime chỉ còn Mongo nhưng tài liệu còn dấu vết DB cũ	Đồng bộ tài liệu môi trường với implementation
Tái sử dụng	BaseService và BaseRepository giảm lặp	Có nguy cơ generic hóa quá mức	Chỉ generic hóa phần lặp thực sự ổn định
Tích hợp ngoài	Cloudinary/OTP được cô lập ở service riêng	Cần quản lý lỗi và retry kỹ hơn ở production	Bổ sung logging/observability sâu hơn nếu đi production lớn

Kết luận của phần kiến trúc là: hệ thống hiện tại phù hợp nhất với mô tả Modular Monolith kết hợp Modular Layered Architecture. Đây là một thiết kế hợp lý cho hiện trạng bài toán và cũng là nền tảng tốt để học tập về phân tầng, service layer, repository, adapter và mô hình hóa C4.

V. MINH CHỨNG PHẦN MỀM

1. Demo sản phẩm

Theo yêu cầu của người dùng, phần ảnh demo hiện tại để trống để có thể chèn thủ công sau khi chạy hệ thống. Các khung dưới đây đã được chuẩn bị sẵn với chú thích rõ ràng.

Hình 5.1. Màn hình đăng nhập

De trong - chen anh man hinh dang nhap sau khi chay frontend / Postman

Hình 5.2. Màn hình quên mật khẩu / OTP

De trong - chen anh man hinh forgot password hoac reset password

Hình 5.3. Màn hình quản lý người dùng

De trong - chen anh danh sach user, profile hoac action quan tri

Hình 5.4. Màn hình lịch trực

De trong - chen anh lich truc, dang ky ca hoac yeu cau doi ca

Hình 5.5. Màn hình upload file

De trong - chen anh upload avatar / file va ket qua tra ve

Hình 5.6. Màn hình báo cáo tổng quan

De trong - chen anh report overview / export CSV-XLSX

2. Minh chứng mã nguồn theo nhóm chức năng

Nhóm chức năng	Vị trí mã nguồn	Ý nghĩa minh chứng
Entrypoint và bootstrap	index.ts, src/server.ts	Chứng minh một app Express duy nhất và pipeline middleware chung
Auth	src/modules/auth/*	Minh chứng đăng ký, đăng nhập, refresh token, đổi mật khẩu, OTP reset password
Users	src/modules/users/*	Minh chứng CRUD người dùng,

		cập nhật profile, avatar, import/export, thống kê
Duty	src/modules/duty/*	Minh chứng lịch trực, đăng ký/hủy ca, yêu cầu đổi ca và duyệt đổi ca
Notifications	src/modules/notifications/*	Minh chứng danh sách thông báo, settings, mark as read, clear all
Files	src/modules/files/*	Minh chứng upload avatar/file, metadata, storage info và cleanup Cloudinary
Reward / Penalty	src/modules/reward-penalties/*	Minh chứng bản ghi thưởng - phạt và thống kê tài chính
Reports	src/modules/reports/*	Minh chứng báo cáo tổng quan và export CSV/XLSX
Shared layer	src/shared/common/*, src/shared/repositories/*	Minh chứng base classes, template method, repository pattern
Persistence	src/types/database.ts, src/database/*	Minh chứng adapter pattern và persistence abstraction

3. Trích đoạn mã nguồn tiêu biểu

3.1. Bootstrap server và middleware chain

```
app.use('/api', camelizeBody, logRequest, wrapJson, appendPaginationHeaders,
parseApiQuery);
setupSwagger(app);
app.use('/api', routes);

async function bootstrap() {
  await initDatabase();
  app.listen(PORT, () => { ... });
}
```

Đoạn mã này minh chứng rõ hệ thống có một pipeline request chung, một route root chung và một bước khởi tạo database tập trung trước khi nhận traffic.

3.2. Router tổng ghép các module

```
router.use('/auth', authRoutes);
router.use('/users', userRoutes);
router.use('/upload', uploadRoutes);
router.use('/files', fileRoutes);
router.use('/notifications', notificationRoutes);
router.use('/duty', dutyRoutes);
router.use('/reward-penalties', rewardPenaltyRoutes);
router.use('/reports', reportRoutes);
```

Đây là bằng chứng trực tiếp của mô hình modular monolith: nhiều module nghiệp vụ được mount trong cùng một Express router.

3.3. Luồng nghiệp vụ đăng nhập trong AuthService

```
const normalizedEmail = this.normalizeEmail(payload.email);
const user = await this.userRepository.findByEmail(normalizedEmail);
```

```
if (!user) throw ApiError.unauthorized('Invalid email or password');
if (!user.isActive) throw ApiError.unauthorized('Account is inactive');
const isMatch = await comparePassword(payload.password, user.password);
await this.userRepository.updateLastLogin(user.id, loginTime);
const token = generateToken(updatedUser.id, updatedUser.lastLogin || loginTime);
```

Trích đoạn này cho thấy business logic xác thực nằm ở service layer thay vì controller, và data access được thực hiện thông qua repository.

3.4. Khung CRUD chung trong BaseService

```
async create(data) {
  const schemaValidation = await this.validateBySchema(data);
  const customValidation = await this.validateCreate(data);
  const transformedData = await this.beforeCreate(data);
  const item = await this.repository.create(transformedData);
  await this.afterCreate(item);
  return { success: true, data: item };
}
```

Trích đoạn này là minh chứng rõ nhất cho Template Method: flow create được cố định nhưng cho phép class con chèn hook ở nhiều bước.

3.5. BaseRepository và DatabaseAdapter

```
class BaseRepository {
  constructor(collection, database = db) {
    this.collection = collection;
    this.database = database;
  }
  async findOne(query) {
    return await this.database.findOne(this.collection, query);
  }
}
```

Đây là minh chứng cho Repository Pattern và Adapter Pattern cùng lúc: repository chuẩn hóa truy cập dữ liệu, còn database được nhìn dưới dạng một contract trừu tượng.

3.6. UploadService rollback khi lỗi ghi DB

```
const asset = await this.storageService.upload(...);
try {
  const fileRecord = await this.buildFileRecord(asset, prepared, options);
  return { ...asset, fileRecord };
} catch (error) {
  await this.storageService.destroy(asset.publicId,
    asset.resourceType).catch(() => null);
  throw error;
}
```

Đoạn này minh chứng tư duy thiết kế an toàn khi tích hợp dịch vụ ngoài: nếu phần ghi metadata thất bại thì file trên Cloudinary được rollback để tránh rác dữ liệu.

4. Lệnh chạy, build và kiểm tra cơ bản

```
npm install
cp .env.example .env
```

```

npm run dev

# Kiem tra app
GET http://localhost:3000/api/health
GET http://localhost:3000/api-docs

# Build production
npm run build
npm start

```

5. Ma trận liên hệ giữa báo cáo và mã nguồn

Nội dung báo cáo	Mã nguồn liên quan
Use case đăng nhập	src/modules/auth/routes/auth.routes.ts, src/modules/auth/controllers/auth.controller.ts, src/modules/auth/services/auth.service.ts
Use case quản lý user	src/modules/users/routes/user.routes.ts, src/modules/users/controllers/user.controller.ts, src/modules/users/services/user.service.ts
Use case lịch trực và đổi ca	src/modules/duty/routes/duty.routes.ts, src/modules/duty/controllers/duty.controller.ts, src/modules/duty/services/duty.service.ts
Use case thông báo	src/modules/notifications/routes/ notification.routes.ts, src/modules/notifications/services/notification.servi ce.ts
Use case upload	src/modules/files/routes/upload.routes.ts, src/modules/files/services/upload.service.ts, src/modules/files/services/cloudinary- storage.service.ts
Mẫu kiến trúc và pattern	src/shared/common/base-service.ts, src/shared/repositories/base.repository.ts, src/types/database.ts, src/database/mongo- database.adapter.ts

Phần minh chứng phần mềm cho thấy các kết luận trong báo cáo không phải suy luận cảm tính mà được rút ra từ các tuyến mã nguồn cụ thể. Đây là điểm quan trọng để bảo vệ báo cáo trước giảng viên hoặc khi nhóm cần chỉnh sửa, cập nhật tài liệu về sau.

PHỤ LỤC A. DANH MỤC ENDPOINT CHÍNH

Module	Method	Path	Mô tả ngắn
Auth	POST	/api/auth/register	Đăng ký tài khoản
Auth	POST	/api/auth/login	Đăng nhập hệ thống
Auth	GET	/api/auth/me	Lấy thông tin người dùng hiện tại
Auth	PUT	/api/auth/change-password	Đổi mật khẩu
Auth	POST	/api/auth/forgot-password	Yêu cầu OTP reset password
Auth	POST	/api/auth/reset-password	Đặt lại mật khẩu bằng OTP
Users	GET	/api/users	Danh sách user có phân trang / filter
Users	POST	/api/users	Tạo user mới
Users	PUT	/api/users/:id	Cập nhật user
Users	DELETE	/api/users/:id	Xóa user
Users	PUT	/api/users/profile	Cập nhật profile người dùng hiện tại
Users	GET	/api/users/stats/summary	Thống kê user
Users	POST	/api/users/import	Import dữ liệu user
Users	GET	/api/users/export	Export dữ liệu user
Duty	GET	/api/duty/week	Lấy lịch trực theo tuần
Duty	POST	/api/duty/slots	Tạo ca trực
Duty	PATCH	/api/duty/slots/:id/register	Đăng ký ca trực
Duty	PATCH	/api/duty/slots/:id/cancel	Hủy ca trực
Duty	POST	/api/duty/swaps	Tạo yêu cầu đổi ca
Duty	PATCH	/api/duty/swaps/:id/decision	Duyệt / từ chối đổi ca
Notifications	GET	/api/notifications	Lấy danh sách thông báo
Notifications	PUT	/api/notifications/settings	Cập nhật cấu hình thông báo
Files	POST	/api/upload/avatar	Upload avatar
Files	POST	/api/upload/general	Upload file chung
Files	GET	/api/upload/file/info	Lấy thông tin file
Files	DELETE	/api/upload/file	Xóa file
Reports	GET	/api/reports/overview	Lấy báo cáo tổng quan
Reports	GET	/api/reports/export	Xuất báo cáo
Reward-Penalties	GET	/api/reward-penalties	Lấy lịch sử thưởng - phạt
Reward-Penalties	POST	/api/reward-penalties	Tạo bản ghi thưởng - phạt

PHỤ LỤC B. TỔNG HỢP THỰC THỂ DỮ LIỆU

Thực thể	Thuộc tính nổi bật	Quan hệ chính
User	name, email, password, role, status, avatar, isActive, lastLogin	Liên kết với Notification, NotificationSetting, FileAsset, DutySlot, RewardPenalty
Notification	userId, title, message, type, category, channel, isRead	Thuộc về một User
NotificationSetting	userId, shiftNotifications, approvalNotifications, emailNotifications, smsNotifications	One-to-one với User
FileAsset	idFile, urlFile, uploadedBy, mimeType, provider, bytes	uploadedBy tham chiếu User
DutySlot	weekStart, shiftDate, shiftLabel, startTime, endTime, capacity, assignedUserIds, status	Nhiều User có thể gắn vào thông qua assignedUserIds
DutySwapRequest	dutySlotId, requesterId, targetUserId, status, decisionNote, approvedBy	Thuộc về DutySlot và tham chiếu nhiều User theo vai trò
RewardPenalty	userId, type, amount, reason, eventDate, createdBy	Tham chiếu User được thưởng/phạt và User tạo bản ghi

PHỤ LỤC C. MA TRẬN BẰNG CHỨNG KIẾN TRÚC VÀ PATTERN

Kết luận	Bảng chứng code	Ý nghĩa
Modular Monolith	index.ts, src/server.ts, src/routes/index.ts	Một runtime chung, một app Express chung, nhiều module nghệp vụ trong cùng codebase
Layered Architecture	routes -> controllers -> services -> repositories	Trách nhiệm được tách theo tầng
Repository Pattern	src/shared/repositories/ base.repository.ts	Chuẩn hóa data access
Template Method	src/shared/common/base- service.ts	CRUD chung + hook mở rộng
Adapter Pattern	src/types/database.ts, src/database/mongo- database.adapter.ts	Tách repository khỏi chi tiết DB
Chain of Responsibility	middleware auth, rbac, validation, parse query	Xử lý request theo chuỗi trách nhiệm
Facade-like UploadService	src/modules/files/services/ upload.service.ts	Che giấu chuỗi xử lý upload phức hợp